Singleton-Pattern

Carsten Gips (HSBI)

TL;DR
Wenn von einer Klasse nur genau ein Objekt angelegt werden kann, nennt man dies auch das "Singleton-Pattern". Dazu muss verhindert werden, dass der Konstruktor aufgerufen werden kann. Üblicherweise "versteckt" man diesen einfach (Sichtbarkeit auf `private` setzen). Für den Zugriff auf die Instanz bietet man eine statische Methode an. Im Prinzip kann man die Instanz direkt beim Laden der Klasse anlegen ("Eager") oder abwarten, bis die Instanz über die statische Methode angefordert wird, und das Objekt erst dann anlegen ("Lazy").
Videos

- VL Singleton-Pattern

---

1 Motivation

```
public enum Fach { IFM, ELM, ARC }
```

```
Logger l = Logger.getLogger(MyClass.class.getName());
```

Von den Enum-Konstanten soll es nur genau eine Instantiierung, also jeweils nur genau ein Objekt geben. Ähnlich war es beim Logging: Für jeden Namen soll/darf es nur einen tatsächlichen Logger (== Objekt) geben.

Dies nennt man "**Singleton Pattern**".

Anmerkung: Im Logger-Fall handelt es sich streng genommen nicht um ein Singleton, da es vom Logger mehrere Instanzen geben kann (wenn der Name sich unterscheidet). Aber jeden Logger mit einem bestimmten Namen gibt es nur einmal im ganzen Programm, insofern ist es doch wieder ein Beispiel für das Singleton-Pattern …

2 Umsetzung: "Eager" Singleton Pattern

Damit man von "außen" keine Instanzen einer Klasse anlegen kann, versteckt man den Konstruktor, d.h. man setzt die Sichtbarkeit auf `private`. Zusätzlich benötigt man eine Methode, die das Objekt zurückliefern kann. Beim Logger war dies beispielsweise der Aufruf `Logger.getLogger("name")`.

Man kann verschiedene Ausprägungen bei der Umsetzung des Singleton Patterns beobachten. Die beiden wichtigsten sind das "Eager Singleton Pattern" und das "Lazy Singleton Pattern". Der Unterschied liegt darin, wann genau das Objekt erzeugt wird: Beim "Eager Singleton Pattern" wird es direkt beim Laden der Klasse erzeugt.

```
public class SingletonEager {
    private static final SingletonEager inst = new SingletonEager();

    // Privater Constructor: Niemand kann Objekte außerhalb der Klasse anlegen
    private SingletonEager() {}

    public static SingletonEager getInst() {
```

```
        return inst;
    }
}
```

Beispiel: singleton.SingletonEager

3 Umsetzung: “Lazy” Singleton Pattern

Beim “Lazy Singleton Pattern” wird das Objekt erst erzeugt, wenn die Instanz tatsächlich benötigt wird (also erst beim Aufruf der `get`-Methode).

```
public class SingletonLazy {
    private static SingletonLazy inst = null;

    // Privater Constructor: Niemand kann Objekte außerhalb der Klasse anlegen
    private SingletonLazy() {}

    public static SingletonLazy getInst() {
        // Thread-safe. Kann weggelassen werden bei Single-Threaded-Gebrauch
        synchronized (SingletonLazy.class) {
            if (inst == null) {
                inst = new SingletonLazy();
            }
        }
        return inst;
    }
}
```

Beispiel: singleton.SingletonLazy

4 Vorsicht!

Sie schaffen damit eine globale Variable!

Da es von der Klasse nur eine Instanz gibt, und Sie sich diese dank der statischen Methode an jeder Stelle im Programm “geben” lassen können, haben Sie in der Praxis eine globale Variable geschaffen. Das kann direkt zu schlechter Programmierung (ver-) führen. Zudem wird der Code schwerer lesbar/navigierbar, da diese Singletons nicht über die Schnittstellen von Methoden übergeben werden müssen.

Nutzen Sie das Pattern **sparsam**.

5 Wrap-Up

Singleton-Pattern: Klasse, von der nur genau ein Objekt instantiiert werden kann

1. Konstruktor “verstecken” (Sichtbarkeit auf `private` setzen)
2. Methode zum Zugriff auf die eine Instanz
3. Anlegen der Instanz beispielsweise beim Laden der Klasse (“Eager”) oder beim Aufruf der Zugriffsmethode (“Lazy”)

6 Zum Nachlesen

- Nystrom (2014, Kap. 6)

Lernziele

- k2: Was ist ein Singleton? Was ist der Unterschied zw. einem Lazy und einem Eager Singleton?
- k3: Anwendung des Singleton-Patterns

Quizzes

- Quiz Singleton-Pattern (ILIAS)

Quellen

Nystrom, R. 2014. *Game Programming Patterns*. Genever Benning. https://github.com/munificent/game-programming-patterns.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 9289c1f (Remove Hugo: remove 'cbox', 2025-04-29)